41

局部性和快速文件系统

UNIX 操作系统刚问世时，UNIX 大师 Ken Thompson 亲自编写了第一个文件系统（file system）。我们把它叫作“老 UNIX 文件系统”，它真的很简单。它的数据结构（data structure）在磁盘上基本是这个样子：



超级块（superblock，S）包含整个文件系统的信息：卷（volume）有多大、有多少个 inode、指向空闲块链表（free list）表头的指针，等等。磁盘上的 inode 区域存放着文件系统的全部 inode。最后，磁盘的大部分空间都被数据块（data block）占据。

老文件系统的好处是简单，而且支持文件系统要提供的基本抽象（abstraction）：文件和目录层次结构（directory hierarchy）。这个易用的系统比以往笨拙的、基于记录的存储系统前进了一大步，而目录层次结构（directory hierarchy）相比早期系统那种较简单的单层结构，也是一个真正的进步。

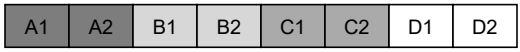
41.1 问题：性能很差

问题在于：性能糟透了。按照伯克利的 Kirk McKusick 和同事们的测量 [MJLF84]，性能一开始就不好，而且随着时间推移越来越差，最后文件系统只发挥出整个磁盘带宽（bandwidth）的 2%！

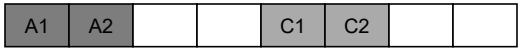
主要问题在于，老 UNIX 文件系统把磁盘当成了随机存取存储器（random-access memory）来对待；数据散落在各处，完全不顾存放数据的介质是磁盘、因而有着实实在在且代价高昂的定位开销。例如，文件的数据块常常离它的 inode 很远，于是每次先读 inode、再读文件的数据块（这是相当常见的操作）时，都要付出一次代价高昂的寻道（seek）。

更糟的是，由于空闲空间（free space）没有得到精心管理，文件系统最终会变得相当**碎片化**（fragmented）。空闲链表（free list）最终会指向散布在磁盘各处的一堆块（block），而分配文件时，只是简单地取下一个空闲块（free block）。结果，访问一个逻辑上连续的文件时，要在磁盘上来回移动，性能因此大幅下降。

例如，设想下面这个数据块区域，里面有四个文件（A、B、C 和 D），每个文件 2 个块：



如果删除 B 和 D，得到的布局是：



可以看到，空闲空间被分成了两段，每段两个块，而不是连续的一整段四个块。假设现在要分配一个大小为四个块的文件 E：



你可以看到发生了什么：E 被分散在磁盘上，因此访问 E 时，无法从磁盘获得峰值（顺序）性能。你得先读 E1 和 E2，然后寻道，再读 E3 和 E4。这种碎片问题在老 UNIX 文件系统中一直存在，而且损害了性能。顺便说一句：这个问题正是磁盘**碎片整理**（defragmentation）工具要解决的；它们重新组织磁盘上的数据，把文件连续存放，并把空闲空间整理成一个或少数几个连续区域，具体做法是移动数据，再改写 inode 等结构来反映这些变化。

另一个问题是：最初的块大小（block size）太小了（512 字节）。因此，从磁盘传输数据天生就效率低下。小块的好处是能把**内部碎片**（internal fragmentation，块内的浪费）降到最低，但不利于传输，因为每个块都可能需要一次定位开销才能访问到。于是，问题来了：

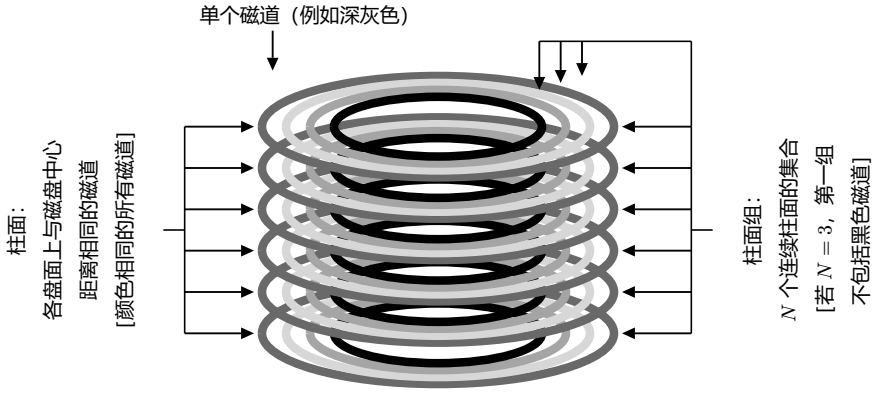
关键问题：
如何组织磁盘上的数据以提高性能
如何组织文件系统的数据结构来提高性能？在这些数据结构之上，需要什么样的分配策略？如何让文件系统“了解磁盘”（disk aware）？

41.2 FFS：解决方案是了解磁盘

伯克利的一个小组决定构建一个更好、更快的文件系统，他们巧妙地把它命名为**快速文件系统**（Fast File System，FFS）。他们的想法是让文件系统的结构和分配策略“了解磁盘”，从而提高性能，他们也正是这么做的。FFS 由此开启了文件系统研究的新时代：作者保留文件系统原有的接口（同样的 API，包括 `open()`、`read()`、`write()`、`close()` 和其他文件系统调用），只改变内部实现，从而为构建新的文件系统铺平了道路，这方面的工作一直延续到今天。几乎所有现代文件系统都遵循现有的接口（从而保持与应用程序的兼容），同时出于性能、可靠性（reliability）或其他原因改变内部实现。

41.3 组织结构：柱面组

第一步是改变磁盘上的结构。FFS 把磁盘划分成若干个**柱面组**（cylinder group）。一个**柱面**（cylinder）是硬盘不同盘面（surface）上、与磁盘中心距离相同的一组磁道（track）；之所以叫柱面（cylinder），是因为它明显很像那个几何形状。FFS 把 N 个连续的柱面（cylinder）合成一组，于是整个磁盘就可以看成由许多柱面组（cylinder group）构成的集合。下面是一个简单的例子，展示了一个有六个盘片（platter）的驱动器最外侧的四个磁道，以及一个由三个柱面组成的柱面组（cylinder group）：



注意，现代驱动器并没有向外提供足够的信息，让文件系统真正了解某个柱面（cylinder）是否正在使用；我们之前讨论过 [AD18a]，磁盘向外提供的是由块组成的逻辑地址空间（logical address space），并向使用者隐藏了它的几何细节。因此，现代文件系统，比如 Linux 的 ext2、ext3 和 ext4，改为把驱动器组织成**块组**（block group），每个块组只是磁盘地址空间（address space）中连续的一段。下图举了一个例子，每 8 个块组成一个块组，而真实的块组包含的块要多得多：



不管叫柱面组（cylinder group）还是块组，这些组都是 FFS 用来提高性能的核心机制。关键在于，把两个文件放在同一个组里，FFS 就能确保先后访问它们时，不会在磁盘上做长距离的寻道。

为了用这些组来存放文件和目录（directory），FFS 需要能把文件和目录放进某个组，并在组里记录关于它们的所有必要信息。为此，FFS 在每个组中都包含了文件系统应有的全部结构，比如 inode 的空间、数据块，以及一些用来记录它们是已分配还是空闲的结构。下面描绘了 FFS 在单个柱面组（cylinder group）中保存的内容：



现在我们来更详细地看看单个柱面组（cylinder group）的各个组成部分。出于可靠性的考虑，FFS 在每个组中都保存一份**超级块**（super block，S）的副本。挂载（mount）文件系统需要超级块；保存多个副本的话，即使有一份损坏了，仍然可以用一份完好的副本来挂载和访问文件系统。

在每个组中，FFS 需要记录这个组的 inode 和数据块是否已分配。每个组都有一个**inode 位图**（inode bitmap，ib）和一个**数据位图**（data bitmap，db），分别为组内的 inode

补充：FFS 中的文件创建

举个例子，想一想创建一个文件时必须更新哪些数据结构；在这个例子中，假设用户创建了一个新文件 `/foo/bar.txt`，文件长度为一个块（4 KB）。这个文件是新的，因此需要一个新的 inode；于是，inode 位图和新分配的 inode 都要写入磁盘。这个文件里还有数据，所以也要为它分配数据块；于是，数据位图（data bitmap）和一个数据块（最终）也要写入磁盘。因此，当前柱面组（cylinder group）至少要发生四次写（回想一下，这些写在真正执行之前，可能会先在内存中缓冲一段时间）。但这还不是全部！特别是，创建新文件时，还必须把文件放进文件系统的层次结构中，也就是说，必须更新目录。具体来说，必须更新父目录（parent directory）`foo`，加入 `bar.txt` 的条目；这次更新可能放得进 `foo` 现有的某个数据块，也可能需要分配一个新块（以及相关的数据位图（data bitmap））。`foo` 的 inode 也必须更新，既要反映目录的新长度，也要更新时间字段（比如最后修改时间）。总之，仅仅创建一个新文件就要做大量工作！下次你创建文件时，也许应该更心怀感激，至少该惊叹于这一切居然运转得这么好。

和数据块承担这项任务。位图（bitmap）是管理文件系统中空闲空间的绝佳方式，因为很容易找到一大块空闲空间并把它分配给文件，或许还能避免老文件系统中空闲链表的一些碎片问题。

最后，**inode** 区域和**数据块**（data block）区域与之前那个非常简单的文件系统（VSFS）中的一样。和往常一样，每个柱面组（cylinder group）的大部分空间都是数据块。

41.4 策略：如何分配文件和目录

有了这样的组结构，FFS 现在要决定如何把文件、目录以及相关的元数据（metadata）放到磁盘上，以提高性能。基本原则很简单：*把相关的东西放在一起*（以及它的推论：*把不相关的东西远远分开*）。

因此，为了遵守这条原则，FFS 必须判断什么是“相关的”，把它们放进同一个块组；反过来，不相关的东西应该放进不同的块组。为了做到这一点，FFS 使用了几条简单的放置启发式规则。

第一条是目录的放置。FFS 的做法很简单：找一个已分配目录数较少（以便在各组之间平衡目录）而空闲 inode 数较多（以便之后能分配一批文件）的柱面组（cylinder group），把目录的数据和 inode 放在这个组里。当然，这里也可以用别的启发式规则（例如把空闲数据块的数量也考虑进去）。

对于文件，FFS 做两件事。第一，它确保（一般情况下）把文件的数据块和它的 inode 分配在同一个组里，从而避免在 inode 和数据之间做长距离寻道（就像老文件系统那样）。第二，它把同一目录下的所有文件，都放在它们所在目录的那个柱面组（cylinder group）里。因此，如果用户创建了四个文件 `/a/b`、`/a/c`、`/a/d` 和 `/b/f`，FFS 会尽量把前三个放得彼此靠近（在同一个组），而把第四个放得远一些（在另外某个组）。

我们来看一个这样分配的例子。在这个例子中，假设每个组只有 10 个 inode 和 10 个数据块（两个数字都小得不现实），三个目录，即根目录（root directory）`/`、`/a` 和 `/b`，以

 © 2008–26, ARPACI-DUSSEAU

及四个文件（/a/c、/a/d、/a/e、/b/f）按照 FFS 的策略放置在这些组中。假设每个普通文件都是两个块大小，目录只有一个数据块。在这个图中，我们用一目了然的符号表示每个文件或目录（即用 / 表示根目录，a 表示 /a，f 表示 /b/f，依此类推）。

```
group inodes      data
    0 /---------  /---------
    1 acde------  accddee---
    2 bf--------  bff-------
    3 ----------  ----------
    4 ----------  ----------
    5 ----------  ----------
    6 ----------  ----------
    7 ----------  ----------
```

注意，FFS 的策略带来了两个好处：每个文件的数据块都靠近这个文件的 inode，同一目录下的文件也彼此靠近（即 /a/c、/a/d 和 /a/e 都在组 1 中，目录 /b 和它的文件 /b/f 在组 2 中彼此靠近）。

作为对比，我们再来看一种 inode 分配策略，它只是把 inode 分散到各组，尽量保证没有哪个组的 inode 表（inode table）很快被填满。最终的分配结果可能是这样的：

```
group inodes      data
    0 /---------  /---------
    1 a---------  a---------
    2 b---------  b---------
    3 c---------  cc--------
    4 d---------  dd--------
    5 e---------  ee--------
    6 f---------  ff--------
    7 ----------  ----------
```

从图中可以看出，这种策略虽然确实让文件和目录的数据靠近各自的 inode，但同一目录下的文件被随意地分散在磁盘各处，因此没有保住基于名字的局部性（locality）。现在访问文件 /a/c、/a/d 和 /a/e 要跨越三个组，而按照 FFS 的方法只需要一个组。

FFS 的策略启发式规则并不是基于对文件系统流量的大量研究，也没有什么特别精妙之处；它们靠的是老派的**常识**（common sense，CS 不就是这个意思吗?）[1]。一个目录中的文件*往往*会被一起访问：想象一下编译一批文件，然后把它们链接成一个可执行文件。正因为存在这种基于命名空间（name space）的局部性，FFS 往往能提高性能，确保相关文件之间的寻道又近又短。

41.5 测量文件的局部性

为了更好地理解这些启发式规则是否合理，我们来分析一些文件系统访问的跟踪记录，看看是否真的存在命名空间局部性（namespace locality）。不知为什么，文献中似乎没有关于这个话题的好研究。

[1]有些人把常识叫作 **horse sense**，尤其是经常和马打交道的人。不过我们觉得，随着“机械马”（也就是汽车）越来越普及，这个说法可能会失传。他们下一步还会发明什么？会飞的机器??!!

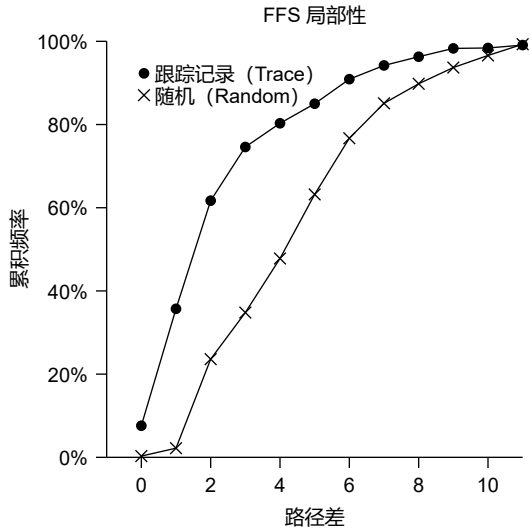


图 41.1：**SEER 跟踪记录中的 FFS 局部性**

具体来说，我们将使用 SEER 跟踪记录 [K94]，分析文件访问在目录树（directory tree）中彼此“相距多远”。例如，如果打开了文件 `f`，然后在跟踪记录中紧接着又打开了它（在打开任何其他文件之前），那么这两次打开在目录树中的距离就是零（因为它们是同一个文件）。如果打开了目录 `dir` 中的文件 `f`（即 `dir/f`），接着打开了同一目录中的文件 `g`（即 `dir/g`），那么这两次文件访问的距离就是一，因为它们在同一个目录下，但不是同一个文件。换句话说，我们的距离度量衡量的是：要找到两个文件的*共同祖先*，需要在目录树中往上走多远；它们在树中离得越近，这个度量值就越小。

图 41.1 展示了在 SEER 集群所有工作站上、在全部跟踪记录中观察到的局部性。图中横轴是差值度量，纵轴是差值为该值的文件打开操作所占的累积百分比。具体来说，从 SEER 跟踪记录（图中标为“Trace”）可以看到，约 7% 的文件访问是访问上一次打开的那个文件，而将近 40% 的文件访问要么是同一个文件，要么是同一目录下的文件（即差值为零或一）。因此，FFS 的局部性假设看来是合理的（至少对这些跟踪记录而言）。

有意思的是，另外还有约 25% 的文件访问，访问的是距离为二的文件。如果用户以多级的方式组织了一组相关的目录，并经常在它们之间来回切换，就会出现这种局部性。例如，如果用户有一个 `src` 目录，并把目标文件（`.o` 文件）构建到 `obj` 目录中，而这两个目录都是主目录 `proj` 的子目录（subdirectory），那么常见的访问模式就是先访问 `proj/src/foo.c`，接着访问 `proj/obj/foo.o`。这两次访问的距离是二，因为 `proj` 是它们的共同祖先。FFS 的策略没有捕捉到这种局部性，因此这类访问之间会有更多的寻道。

作为对比，图中还给出了一个“随机”（Random）跟踪记录的局部性。随机跟踪记录的生成方法是：从现有的 SEER 跟踪记录中按随机顺序选取文件，再计算这些随机顺序的访问之间的距离度量。可以看到，和预期的一样，随机跟踪记录中的命名空间局部性（namespace

locality）更少。不过，由于所有文件最终都有一个共同祖先（比如根目录），所以还是有一些局部性，因此随机跟踪记录可以作为一个有用的比较基准。

41.6 大文件的例外规则

在 FFS 中，文件放置的一般策略有一个重要的例外，它针对的是大文件。如果没有一条不同的规则，一个大文件就会把它最先放入的那个块组完全填满，也许还会填满其他块组。这样填满一个块组是不可取的，因为它会妨碍随后的“相关”文件放进这个块组，从而可能损害文件访问的局部性。

因此，对于大文件，FFS 是这样做的：在第一个块组中分配了一定数量的块之后（例如 12 个块，也就是一个 inode 中直接指针的数量），FFS 把文件的下一个“大”块（例如第一个间接块所指向的那些块）放到另一个块组中（可能会选一个利用率（utilization）较低的组）。然后，文件的再下一个大块（chunk）又放到另一个块组中，依此类推。

我们来看几张图，更好地理解这条策略。如果没有这条大文件例外规则，一个大文件会把它的所有块都放在磁盘的同一个区域。我们来研究一个小例子：在一个配置为每组 10 个 inode、40 个数据块的 FFS 中，有一个 30 个块的文件（`/a`）。下面是没有大文件例外规则时 FFS 的情形：

```
group inodes      data
    0 /a--------  /aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a---------
    1 ----------  ---------- ---------- ---------- ----------
    2 ----------  ---------- ---------- ---------- ----------
```

从图中可以看到，`/a` 占满了组 0 的大部分数据块，而其他组仍然是空的。如果现在在根目录（`/`）中再创建一些别的文件，组里就没有多少空间存放它们的数据了。

有了大文件例外规则（这里设为每个大块五个块），FFS 就会把这个文件分散到各个组中，这样任何一个组的利用率都不会太高：

```
group inodes      data
    0 /a--------  /aaaaa---- ---------- ---------- ----------
    1 ----------  aaaaa----- ---------- ---------- ----------
    2 ----------  aaaaa----- ---------- ---------- ----------
    3 ----------  aaaaa----- ---------- ---------- ----------
    4 ----------  aaaaa----- ---------- ---------- ----------
    5 ----------  aaaaa----- ---------- ---------- ----------
    6 ----------  ---------- ---------- ---------- ----------
```

精明的读者（就是你）会注意到，把一个文件的块分散到磁盘各处会损害性能，尤其是在顺序访问文件这种比较常见的情况下（例如用户或应用程序按顺序读取第 0 块到第 29 块）。你说得没错，我们精明的读者！不过，通过仔细选择大块（chunk）的大小，就可以解决这个问题。

具体来说，如果大块（chunk）足够大，文件系统的大部分时间就会花在从磁盘传输数据上，而在大块之间寻道只花（相对）很少的时间。像这样通过每付出一次开销多做一些工作来降低开销的过程，叫作**平摊**（amortization），这是计算机系统中的一种常用技术。

我们来看一个例子：假设磁盘的平均定位时间，即寻道加旋转的时间，是 10 ms，再假设磁盘以 40 MB/s 的速率传输数据。如果你的目标是一半时间用来在大块（chunk）之间寻道，一半时间用来传输数据（从而达到磁盘峰值性能的 50%），那么每做 10 ms 的定位，就需要花 10 ms 传输数据。于是问题变成：大块（chunk）要多大，才能让传输花 10 ms？很简单，用我们的老朋友数学就行，特别是在磁盘一章中提到的量纲分析（dimensional analysis）[AD18a]：

$$\frac{40\cancel{MB}}{\cancel{秒}} \cdot \frac{1024\ \text{KB}}{1\cancel{MB}} \cdot \frac{1\cancel{秒}}{1000\cancel{ms}} \cdot 10\cancel{ms} = 409.6\ \text{KB} \tag{41.1}$$

这个公式的意思基本上是：如果以 40 MB/s 的速率传输数据，那么每寻道一次，只需传输 409.6 KB，就能做到一半时间寻道、一半时间传输。类似地，你可以算出要达到峰值带宽的 90% 需要多大的大块（结果大约是 3.6 MB），甚至达到峰值带宽的 99% 需要多大的大块（39.6 MB!）。可以看到，你想越接近峰值，这些大块（chunk）就越大（这些值的曲线见图 41.2）。

不过，FFS 并没有用这种计算来把大文件分散到各个组中。它采用了一种简单的方法，依据的是 inode 本身的结构：最前面的 12 个直接块（direct block）和 inode 放在同一个组中；之后的每个间接块（indirect block），以及它指向的所有块，都放到不同的组中。块大小为 4 KB、磁盘地址（disk address）为 32 位时，这个策略意味着文件每 1024 个块（4 MB）放在一个单独的组中，唯一的例外是文件开头由直接指针（direct pointer）指向的那 48 KB。

注意，磁盘驱动器（disk drive）的发展趋势是传输速率提高得相当快，因为磁盘制造商擅长在同样的盘面上塞进更多的位；但驱动器与寻道相关的机械方面（磁盘臂（disk arm）的速度和旋转速率）却提高得相当慢 [P98]。这意味着，随着时间的推移，机械开销会变得相对越来越昂贵，因此，要平摊（amortize）这些开销，就必须在两次寻道之间传输更多的数据。

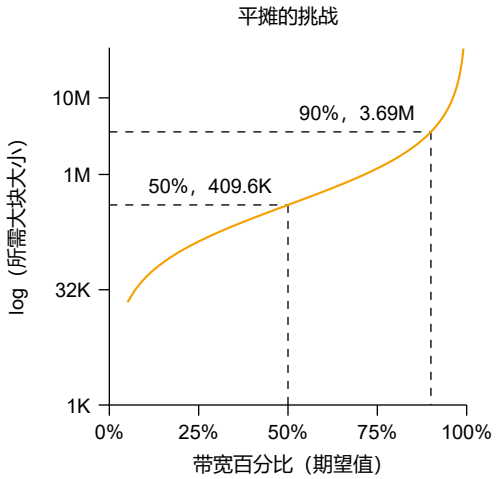


图 41.2：**平摊：大块要多大才行？**

 © 2008–26, ARPACI-DUSSEAU

41.7 关于 FFS 的其他几件事

FFS 还引入了其他一些创新。特别是，设计者非常担心如何容纳小文件；事实证明，当时很多文件只有 2 KB 左右，而使用 4 KB 的块虽然有利于传输数据，但空间效率却不太好。因此，在一个典型的文件系统中，这种**内部碎片**（internal fragmentation）可能导致大约一半的磁盘空间被浪费。

FFS 设计者想到的解决办法很简单，也解决了问题。他们决定引入**子块**（sub-block），也就是文件系统可以分配给文件的 512 字节的小块。这样，如果你创建一个小文件（比如 1 KB 大小），它会占用两个子块（sub-block），从而不会浪费一整个 4 KB 的块。随着文件增长，文件系统会继续给它分配 512 字节的块，直到它攒满 4 KB 的数据。这时，FFS 会找一个 4 KB 的块，把这些子块（sub-block）复制进去，并释放这些子块供以后使用。

你可能会发现，这个过程效率不高，需要文件系统做大量额外的工作（特别是，要做大量额外的 I/O 来执行复制）。你又说对了！因此，FFS 通常通过修改 `libc` 库来避免这种最糟糕的情况：库会缓冲写操作，然后以 4 KB 的块为单位把它们交给文件系统，这样在大多数情况下就完全不必用到子块（sub-block）这个特殊机制了。

FFS 引入的第二个巧妙之处，是一种为性能而优化的磁盘布局。在那个年代（SCSI 和其他更现代的设备接口出现之前），磁盘要简单得多，需要主机 CPU 更直接地控制它们的操作。当一个文件放在磁盘的连续扇区（sector）上时（如图 41.3 左边所示），FFS 就出现了问题。

具体来说，问题出现在顺序读取的时候。FFS 会先发出读块 0 的请求；等这次读完成、FFS 再发出读块 1 的请求时，已经太晚了：块 1 已经转过了磁头（disk head），现在读块 1 就要等上整整一圈。

FFS 用一种不同的布局解决了这个问题，如图 41.3 右边所示。通过（在这个例子中）每隔一个块放一个块，FFS 就有足够的时间在下一个块转过磁头之前发出请求。事实上，FFS 足够聪明，能针对某个具体的磁盘算出布局时应该跳过多少个块，才能避免额外的旋转；这种技术叫作**参数化**（parameterization），因为 FFS 会测出磁盘具体的性能参数，并用它们来决定确切的交错布局方案。

你可能会想：这个方案说到底也没那么好。事实上，用这种布局只能得到峰值带宽的 50%，因为你必须绕每个磁道转两圈，才能把每个块各读一次。幸好，现代磁盘要聪明得多：它们会在内部把整个磁道读进来，缓冲在磁盘内部的缓存（cache）里（正因如此，这个缓

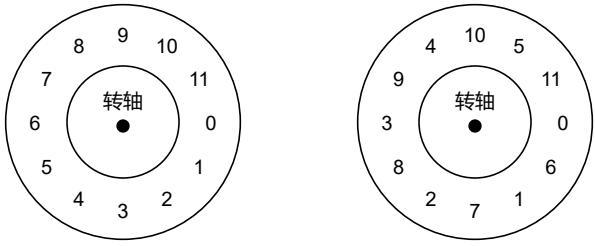


图 41.3：**FFS：标准放置与参数化放置**

提示：让系统好用

FFS 带给我们的最基本的一课，大概是：它不仅引入了“了解磁盘”的布局这个在概念上很好的想法，还加入了一些单纯让系统更好用的功能。长文件名（long file name）、符号链接（symbolic link），以及以原子方式完成的重命名操作，都提高了系统的实用性；虽然很难就它们写一篇研究论文（想象一下读一篇 14 页的论文，题目叫《符号链接：硬链接失散多年的表亲》），但正是这些小功能让 FFS 更有用，因此很可能也提高了它被采用的机会。让系统好用，往往和系统深层的技术创新一样重要，甚至更重要。

存常被称为**磁道缓冲区**，track buffer）。之后再读这个磁道时，磁盘只需从缓存中返回所需的数据。因此，文件系统再也不用操心这些极其底层的细节了。只要设计得当，抽象和更高层的接口会是好东西。

FFS 还做了其他一些易用性方面的改进。它是最早支持**长文件名**（long file name）的文件系统之一，从而让文件系统中可以使用更有表达力的名字，而不再是传统的固定长度的做法（比如 8 个字符）。此外，它还引入了一个新概念，叫作**符号链接**（symbolic link）。正如前一章 [AD18b] 讨论过的，硬链接（hard link）有局限：它们既不能指向目录（因为担心在文件系统层次结构中引入环路），也只能指向同一个卷中的文件（即 inode 号（inode number）必须仍然有意义）。符号链接让用户可以为系统中的任何其他文件或目录创建一个“别名”，因此要灵活得多。FFS 还引入了用于重命名文件的原子 `rename()` 操作。在基础技术之外的这些易用性改进，很可能也为 FFS 赢得了更强大的用户群。

41.8 小结

FFS 的问世是文件系统历史上的一个分水岭，它让人们清楚地认识到，文件管理问题是操作系统中最有意思的问题之一，并展示了如何着手对付最重要的设备：硬盘。从那以后，人们开发了数以百计的新文件系统，但直到今天，许多文件系统仍然从 FFS 身上汲取灵感（例如，Linux 的 ext2 和 ext3 显然就是它在思想上的后代）。当然，所有现代系统都吸取了 FFS 的主要教训：把磁盘当磁盘来对待。

 © 2008–26, ARPACI-DUSSEAU

参考文献

[AD18a] "Operating Systems: Three Easy Pieces" (Chapter: Hard Disk Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2018. *There is no way you should be reading about FFS without having first understood hard drives in some detail. If you try to do so, please instead go directly to jail; do not pass go, and, critically, do not collect 200 much-needed simoleons.*

[AD18b] "Operating Systems: Three Easy Pieces" (Chapter: File System Implementation) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau . Arpaci-Dusseau Books, 2018. *As above, it makes little sense to read this chapter unless you have read (and understood) the chapter on file system implementation. Otherwise, we'll be throwing around terms like "inode" and "indirect block" and you'll be like "huh?" and that is no fun for either of us.*

[K94] "The Design of the SEER Predictive Caching System" by G. H. Kuenning. MOBICOMM '94, Santa Cruz, California, December 1994. *According to Kuenning, this is the best overview of the SEER project, which led to (among other things) the collection of these traces.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, 2:3, August 1984. *McKusick was recently honored with the IEEE Reynold B. Johnson award for his contributions to file systems, much of which was based on his work building FFS. In his acceptance speech, he discussed the original FFS software: only 1200 lines of code! Modern versions are a little more complex, e.g., the BSD FFS descendant now is in the 50-thousand lines-of-code range.*

[P98] "Hardware Technology Trends and Database Opportunities" by David A. Patterson. Keynote Lecture at SIGMOD '98, June 1998. *A great and simple overview of disk technology trends and how they change over time.*

 © 2008–26, ARPACI-DUSSEAU

作业（模拟）

这一节介绍 `ffs.py`，一个简单的 FFS 模拟器，你可以用它更好地理解基于 FFS 的文件和目录分配是如何工作的。模拟器的运行方法详见 README。

问题

1. 查看文件 `in.largefile`，然后用选项 `-f in.largefile` 和 `-L 4` 运行模拟器。后一个选项把大文件例外规则设为 4 个块。最终的分配结果会是什么样子？用 `-c` 运行来检查一下。
2. 现在用 `-L 30` 运行。你预计会看到什么？同样，打开 `-c` 看看你猜得对不对。你还可以用 `-S` 准确地看到哪些块分配给了文件 `/a`。
3. 现在我们来计算这个文件的一些统计量。第一个我们称之为 *filespan*，它是文件的任意两个数据块之间、或者 inode 与任意一个数据块之间的最大距离。计算 `/a` 的 filespan。运行 `ffs.py -f in.largefile -L 4 -T -c` 看看它是多少。再用 `-L 100` 做同样的事。随着大文件例外参数从小值变到大值，你预计 filespan 会有什么不同？
4. 现在我们来看一个新的输入文件 `in.manyfiles`。你认为 FFS 的策略会如何把这些文件分布到各个组中？（可以用 `-v` 运行，看看创建了哪些文件和目录，或者直接 `cat in.manyfiles`。）用 `-c` 运行模拟器，看看你猜得对不对。
5. 评价 FFS 的一个指标叫作 *dirspan*。这个指标计算的是某个目录中文件的分散程度，具体来说，是该目录中所有文件的 inode 和数据块，与目录本身的 inode 和数据块之间的最大距离。用 `in.manyfiles` 和 `-T` 选项运行，计算三个目录的 dirspan。用 `-c` 运行来检查。FFS 在减小 dirspan 方面做得怎么样？
6. 现在把每个组的 inode 表大小改成 5（`-i 5`）。你认为这会如何改变文件的布局？用 `-c` 运行，看看你猜得对不对。它对 dirspan 有什么影响？
7. FFS 应该把新目录的 inode 放在哪个组中？默认的（模拟器）策略会找空闲 inode 最多的那个组。另一种策略会找空闲 inode 最多的若干个组。例如，如果用 `-A 2` 运行，分配新目录时，模拟器会把组两两成对地看，挑出最适合分配的那一对。运行 `./ffs.py -f in.manyfiles -i 5 -A 2 -c`，看看用这种策略时分配结果如何变化。它对 dirspan 有什么影响？为什么这种策略可能是好的？
8. 最后一个要研究的策略变化与文件碎片有关。运行 `./ffs.py -f in.fragmented -v`，看看你能否预测剩下的那些文件是如何分配的。用 `-c` 运行来确认你的答案。文件 `/i` 的数据布局有什么有意思的地方？为什么说它有问题？
9. 一种新策略，我们称之为*连续分配*（`-C`），它尽量保证每个文件都连续分配。具体来说，用 `-C n` 时，文件系统在分配一个块之前，会先确保组内有 `n` 个连续的空闲块。运行 `./ffs.py -f in.fragmented -v -C 2 -c` 看看有什么不同。随着传给 `-C` 的参数增大，布局会怎样变化？最后，`-C` 对 filespan 和 dirspan 有什么影响？